

VirtuEx 성능 개선 감사 보고서 (3차)

VirtuEx Performance Improvement Audit Report (3rd)

- 감사일: 2026-03-15
- 감사 범위: 프론트엔드 렌더링/데이터 페칭, 백엔드 API 응답 시간, WebSocket 처리, 레이스 게임 3D 렌더링 성능
- 감사 방법: 소스 코드 정적 분석, 렌더링 파이프라인 추적, 메모리 패턴 분석, 번들 구조 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 프론트엔드 렌더링	0	2	3	2	7
B. 프론트엔드 데이터 페칭	0	2	2	1	5
C. 백엔드 쿼리 성능	1	2	2	0	5
D. WebSocket/실시간	0	1	2	1	4
E. 3D 렌더링 (레이싱)	0	2	3	2	7
F. 메모리/캐시 관리	0	1	2	1	4
G. 빌드/번들 최적화	0	1	1	1	3
합계	1	11	15	8	35

A. 프론트엔드 렌더링 — 7건

[FR-H-01] 리더보드 FLIP 애니메이션 리플로우 강제 (High)

현상: 리더보드 순위 변동 애니메이션에서 `void el.offsetHeight` 로 리플로우를 강제 트리거. 다수의 행이 동시에 변경될 경우 레이아웃 스래싱(layout thrashing) 발생 **위치:** `frontend/src/app/(main)/leaderboard/page.tsx:236` **영향:** 참여자 20명 이상에서 리패치 시 프레임 드롭 가능 **권장:** `requestAnimationFrame` 배치 처리 또는 CSS `will-change` 속성 활용으로 개선 **심각도:** High

[FR-H-02] 토스트 컨테이너 불필요 리렌더링 (High)

현상: `ToastContainer` 가 `useToastStore` 전체 `toasts` 배열을 구독하여, 단일 토스트 추가/제거 시에도 전체 컨테이너 리렌더링 **위치:** `frontend/src/components/ui/ToastContainer.tsx:31-32` **영향:** 토스트가 빈번하게 발생하는 거래 페이지에서 불필요한 렌더 사이클 **권장:** `memo` 가 적용되어 있으나, 개별 토스트를 별도 컴포넌트로 분리하여 리렌더 범위 최소화 **심각도:** High

[FR-M-01] 대시보드 WebSocket 배치 타이머 500ms 지연 (Medium)

현상: WebSocket 가격 업데이트를 500ms 배치로 처리하여 시세 표시에 최대 0.5초 지연 발생 **위치:** `frontend/src/app/(main)/dashboard/page.tsx:216-226` **영향:** 급변동 시장에서 사용자가 지연된 가격을 볼 수 있음 **권장:** 배치 간격을 200-300ms로 축소하거나 `debounce` 전략 적용 **심각도:** Medium

[FR-M-02] 뉴스 목록 필터링 — 전체 재계산 (Medium)

현상: 검색어/날짜 필터 변경 시 `filteredItems` `useMemo`가 전체 200건을 매번 필터링 위치:

`frontend/src/app/(main)/news/page.tsx:121-132` 영향: 필터 입력 시 키스트로크마다 200건 배열 순회 권장: 검색어 `debounce(300ms)` 적용으로 불필요한 필터 연산 감소 심각도: Medium

[FR-M-03] AI 분석 모달 스크롤 잠금 혹은 (Medium)

현상: `useScrollLock(aiModalOpen)` 혹은 모달 열림 시 `body`에 `overflow: hidden` 적용. 모달 닫힘 시 스크롤 위치 복원 로직이 없어 UX 저하 가능 위치: `frontend/src/app/(main)/dashboard/page.tsx:75` , `news/page.tsx:64` 영향: 긴 페이지에서 모달 닫힘 후 스크롤 위치 초기화 위험 권장: 스크롤 위치 저장/복원 로직 추가 심각도: Medium

[FR-L-01] 리더보드 `prevRankMap` ref 메모리 누수 위험 (Low)

현상: `prevRankMap.current` 가 페이지 언마운트 시 자동 정리되지만, 장시간 사용 시 이전 사용자 ID가 계속 축적 위치:

`frontend/src/app/(main)/leaderboard/page.tsx:206` 권장: 리패치 시 현재 데이터에 없는 이전 ID 정리 로직 추가 심각도: Low

[FR-L-02] CSS `@import` 폰트 로딩 — 렌더링 차단 (Low)

현상: HUD 컴포넌트에서 CSS `@import url()` 방식으로 Google Fonts를 런타임에 로드. 첫 렌더링 시 FOUT(Flash of Unstyled Text) 발생 위치: `racing/src/game/ui/HUD.tsx:407` 권장: `<link rel="preload">` 또는 폰트 자체 호스팅으로 전환 심각도: Low

B. 프론트엔드 데이터 페칭 — 5건

[FD-H-01] AI 분석 뉴스 200건 일괄 조회 (High)

현상: AI 분석 시 `/api/news?limit=200` 으로 200건을 한 번에 조회 후 클라이언트에서 50건으로 필터/슬라이싱. 네트워크 대역폭 낭비 위치: `frontend/src/app/(main)/dashboard/page.tsx:154` , `news/page.tsx:157` 영향: 모바일 네트워크에서 1-3초 추가 지연 가능 권장: 백엔드에서 `since` 파라미터 지원하여 최근 24시간 뉴스만 서버 측 필터링 심각도: High

[FD-H-02] 리더보드 10초 폴링 — 서버 부하 누적 (High)

현상: `refetchInterval: 10000` 으로 모든 리더보드 페이지 접속자가 10초마다 API 호출. 동시 접속 100명 시 분당 600회 요청 위치: `frontend/src/hooks/useLeaderboard.ts:70` 영향: `portfolio` 서비스 + `market-data` 서비스에 지속적 부하 권장: 페이지 포커스 시만 리패치(`refetchOnWindowFocus`) + `WebSocket` 기반 푸시 알림으로 전환 심각도: High

[FD-M-01] 대시보드 초기 로딩 워터폴 (Medium)

현상: `useMarketPrices` → `useAssets` → `usePeriodChanges` 순차 로딩 패턴. 초기 데이터 표시까지 3개 API 완료 대기 위치: `frontend/src/app/(main)/dashboard/page.tsx:52-103` 영향: 초기 로딩 시간 증가 (각 API 200-500ms × 3) 권장: `React Suspense` 또는 `Promise.all` 병렬 로딩 패턴 적용 심각도: Medium

[FD-M-02] 관심종목 위치리스트 비인증 시 불필요 쿼리 (Medium)

현상: 비인증 사용자에게도 `useWatchlist()` 혹은 실행되어 401 에러 발생 가능 위치:

`frontend/src/app/(main)/dashboard/page.tsx:106` 영향: 네트워크 에러 노출 및 콘솔 경고 권장: `enabled: isAuthenticated` 옵션으로 조건부 쿼리 실행 심각도: Medium

[FD-L-01] 기간별 등락을 캐시 키 세분화 부재 (Low)

현상: `usePeriodChanges(period)` 호출 시 `period` 변경마다 새 요청 발생. 이전 `period` 데이터 캐시가 유지되거나 `stale` 판단 기준 미설정 **위치:** `frontend/src/hooks/useMarket.ts` (추정) **권장:** `staleTime` 설정으로 동일 `period` 재요청 방지 **심각도:** Low

C. 백엔드 쿼리 성능 — 5건

[BQ-C-01] 리더보드 전체 계좌 스캔 — $O(N)$ 가격 조회 (Critical)

현상: `getLeaderboard()` 메서드가 `Account.findMany()` 로 전체 계좌를 조회한 후, 모든 보유 종목의 시장 가격을 한 번에 가져옴. 계좌 수 $\times$ 보유 종목 수에 비례하여 연산량 증가 **위치:**

`backend/services/portfolio/src/domain/services/balance.service.ts:856-974` **영향:** 계좌 100개, 종목 50개 기준으로 5000회 가격 매핑 연산. 10초 폴링과 결합 시 CPU 부하 심각 **권장:** 리더보드 전용 캐시(30초 TTL) 도입 또는 점진적 업데이트 전략 적용 **심각도:** Critical

[BQ-H-01] 4개 `groupBy` 쿼리 병렬 실행 — 트랜잭션 테이블 풀스캔 (High)

현상: `Promise.all` 로 4개 `groupBy` 쿼리(`holdings`, `deposits`, `withdrawals`, `trades`)를 병렬 실행하지만, 각 쿼리가 트랜잭션 테이블을 `userId IN (...)` 조건으로 조회. 인덱스 없으면 풀스캔 **위치:**

`backend/services/portfolio/src/domain/services/balance.service.ts:862-883` **영향:** 트랜잭션 10만건 이상에서 응답 시간 1초 초과 가능 **권장:** `userId + type` 복합 인덱스 확인 및 필요 시 추가 **심각도:** High

[BQ-H-02] 환율 API 매 리더보드 호출 시 외부 조회 (High)

현상: `getExchangeRate()` 가 리더보드 호출마다 실행. 5초 캐시가 있지만 동시 요청 시 캐시 미스 가능 **위치:**

`backend/services/portfolio/src/domain/services/balance.service.ts:904` **영향:** 외부 API 레이트 리미트 초과 위험 **권장:** 환율을 더 긴 TTL(60초)로 캐싱하거나 별도 스케줄러로 주기적 업데이트 **심각도:** High

[BQ-M-01] Decimal 연산 반복 — 루프 내 객체 생성 (Medium)

현상: `accounts` 루프 내에서 매 반복마다 `new Decimal()` 객체를 여러 개 생성. GC 부하 증가 **위치:**

`backend/services/portfolio/src/domain/services/balance.service.ts:911-946` **영향:** 계좌 100개 $\times$ 보유 10종목 = 1000개 이상 `Decimal` 객체 생성/소멸 **권장:** 가격 비교만 필요한 경우 `native Number` 사용, 정밀도 필요 시 `Decimal` 유지 **심각도:** Medium

[BQ-M-02] 리더보드 정렬 후 슬라이싱 — 전체 정렬 비용 (Medium)

현상: 전체 `results`를 정렬한 후 `slice(0, limit)` 로 상위만 추출. 힙 정렬 등 부분 정렬 알고리즘이 더 효율적 **위치:**

`backend/services/portfolio/src/domain/services/balance.service.ts:956` **영향:** 계좌 1000개 기준 $O(n \log n)$ 정렬 vs $O(n \log k)$ 부분 정렬 **권장:** 현재 규모에서는 영향 미미하나, 사용자 증가 시 부분 정렬 고려 **심각도:** Medium

D. WebSocket/실시간 — 4건

[WS-H-01] 가격 업데이트 배치 처리 시 메모리 누수 (High)

현상: `pendingPricesRef.current` 에 누적된 가격 데이터가 타이머 발화 전까지 메모리에 유지. 구독 종목이 많을 경우 배치 크기 증가 **위치:** `frontend/src/app/(main)/dashboard/page.tsx:213-227` **영향:** 50종목 구독 시 500ms마다 50개 업데이트 객체 생성/해제 **권장:** 이전 가격과 동일한 경우 업데이트 스킵 로직 추가 **심각도:** High

[WS-M-01] WebSocket 구독 심볼 변경 시 재연결 (Medium)

현상: `assets` 배열 변경 시 `symbols` 매모 값이 변경되어 WebSocket 구독이 재설정될 수 있음 **위치:**

`frontend/src/app/(main)/dashboard/page.tsx:210-228` **영향:** `assets` 정렬 변경만으로도 WebSocket 재연

결 트리거 권장: 심볼 목록을 정렬된 문자열로 비교하여 실제 변경 시만 재구독 심각도: Medium

[WS-M-02] 실시간 토스트 MAX_TOASTS 초과 시 기존 토스트 폐기 (Medium)

현상: 5개 제한 초과 시 가장 오래된 토스트를 즉시 제거. 사용자가 읽지 못한 중요 알림(거래 체결 등) 유실 가능 위치:

frontend/src/stores/liveToast.ts:62-63 영향: 연속 거래 체결 시 초기 알림 미확인 권장: 중요도 기반 토스트 우선순위 시스템 도입 심각도: Medium

[WS-L-01] 토스트 타이머 cleanup 부재 (Low)

현상: addToast 내 setTimeout 핸들이 저장되지 않아 컴포넌트 언마운트 시 정리되지 않음. 메모리 누수는 아니지만 잔여 콜백 실행 위치: frontend/src/stores/toast.ts:60-62, frontend/src/stores/liveToast.ts:65-67 영향: 페이지 전환 시 이전 페이지의 토스트 제거 콜백이 빈 배열에 대해 실행 권장: timer ID를 Map으로 관리하고 removeToast 시 clearTimeout 심각도: Low

E. 3D 렌더링 (레이싱) — 7건

[3D-H-01] 트랙 마킹 이중 렌더링 — 고가/지상 분리 후 geometry 개수 2배 (High)

현상: 고가도로 라인 버그 수정으로 center line, edge line, kerbs를 지상/고가 각각 별도 geometry로 렌더링. 고가 구간이 있는 트랙에서 드로우콜 10개 → 20개로 증가 위치: racing/src/game/world/Roads.tsx 영향: 복잡한 트랙에서 GPU 드로우콜 증가로 프레임 드롭 가능 권장: 단일 geometry에 커스텀 shader material로 고도별 가시성 제어, 또는 두 레벨 geometry를 머지 심각도: High

[3D-H-02] useFrame 내 매 프레임 store.getState() 호출 (High)

현상: CarModel.tsx 의 useFrame 콜백에서 매 프레임마다 useGameStore.getState() 호출. 60fps 기준 초당 60회 store 접근 위치: racing/src/game/vehicle/CarModel.tsx:102 영향: 가비지 컬렉션 압박 (Zustand getState 자체는 가벼우나 패턴 확산 시 누적) 권장: 필요한 값만 useRef로 캐싱하고 store 구독은 최소화 심각도: High

[3D-M-01] 교량 기둥 개별 메시 렌더링 — InstancedMesh 미사용 (Medium)

현상: 교량 기둥을 각각 개별 <mesh> 컴포넌트로 렌더링. 기둥이 10쌍(20개)이면 20회 드로우콜 위치: racing/src/game/world/Roads.tsx:469-482 영향: 복잡한 트랙에서 불필요한 드로우콜 증가 권장: InstancedMesh 로 전환하여 단일 드로우콜로 처리 심각도: Medium

[3D-M-02] CanvasTexture 매 CarModel 마운트 시 재생성 (Medium)

현상: LicensePlate 컴포넌트의 useMemo 가 CarModel 마운트마다 canvas 기반 텍스처를 새로 생성 위치: racing/src/game/vehicle/CarModel.tsx:175-197 영향: 리셋/재시작 시 불필요한 canvas 그리기 연산 권장: 텍스처를 모듈 레벨 싱글톤으로 캐싱 심각도: Medium

[3D-M-03] 트랙 geometry 리샘플링 — Math.ceil(segLen) 과도한 샘플 (Medium)

현상: buildCenterLine 에서 세그먼트를 1m 간격으로 리샘플링. 긴 직선(100m)은 100개 샘플 생성. 시각적으로 10m 간격이면 충분 위치: racing/src/game/world/Roads.tsx:78 영향: 66포인트 트랙의 경우 수천 개의 vertex 생성 권장: 리샘플링 간격을 적응형으로 변경 (직선은 길게, 곡선은 짧게) 심각도: Medium

[3D-L-01] PostProcessing 셰이더 — 전체 화면 블룸 비용 (Low)

현상: 포스트 프로세싱에서 블룸 효과가 전체 화면에 적용. 저사양 기기에서 프레임 드롭 가능 위치:

racing/src/game/effects/PostProcessing.tsx 영향: 모바일/저사양 GPU에서 10-20% 성능 저하 권장: 품질 설정에 따른 블룸 비활성화 옵션 제공 심각도: Low

[3D-L-02] 6개 신규 트랙 추가 — 초기 메모리 사용량 증가 (Low)

현상: 12개 트랙의 포인트 데이터가 모두 메모리에 상주. 66포인트 트랙 × 12개 = ~1584개 좌표쌍 위치:

racing/src/maps/tracks.ts 영향: 실제 메모리 영향은 미미하나 (< 100KB), 트랙 수 증가 시 동적 로딩 고려 권장: 현재 규모에서는 무시 가능. 20개 이상 시 lazy import 검토 심각도: Low

F. 메모리/캐시 관리 — 4건

[MC-H-01] 시장가격 5초 인메모리 캐시 — 동시 요청 경쟁 조건 (High)

현상: fetchMarketPrices 캐시가 5초 TTL로 설정되어 있으나, 동시 요청 시 캐시 미스로 중복 외부 API 호출 발생 가능 위치: backend/services/portfolio/src/domain/services/balance.service.ts:991 영향: getPortfolioValuation + getLeaderboard 동시 호출 시 이중 가격 조회 권장: singleflight 패턴 적용 — 동일 키에 대한 중복 요청을 하나로 병합 심각도: High

[MC-M-01] Zustand 토스트 배열 불변 업데이트 — 스프레드 복사 비용 (Medium)

현상: 토스트 추가/제거 시 [...s.toasts, newToast] 패턴으로 전체 배열 복사. 5개 토스트 × 초당 2-3회 업데이트 시 미미하나 패턴 확산 시 비용 증가 위치: frontend/src/stores/toast.ts:57 , frontend/src/stores/liveToast.ts:62 영향: 현재 규모에서는 무시 가능 권장: 현재 규모 유지. 향후 immer 미들웨어 도입 시 구조 공유 활용 심각도: Medium

[MC-M-02] 레이싱 GLB 모델 프리로드 — 메모리 상주 (Medium)

현상: useGLTF.preload('/models/car.glb') 호출로 GLB 모델이 모듈 로드 시점에 즉시 다운로드/파싱. 게임 미실행 시에도 메모리 점유 위치: racing/src/game/vehicle/CarModel.tsx:284 영향: 차량 모델 크기에 따라 수 MB 메모리 상시 점유 권장: 게임 켜진 마운트 시점에만 프리로드하도록 지연 심각도: Medium

[MC-L-01] 레이싱 재사용 텔레메트리 객체 — GC 부담 최소화 확인 (Low)

현상: _telemetry 객체를 모듈 레벨에서 재사용하여 매 프레임 객체 할당 방지. 좋은 패턴이나 객체 내 배열(position)이 매 프레임 갱신 위치: racing/src/game/vehicle/VehicleController.tsx:59-66 영향: 배열 요소만 변경하므로 GC 부담 없음. 양호한 상태 권장: 현재 패턴 유지 (이미 최적) 심각도: Low (정보)

G. 빌드/번들 최적화 — 3건

[BO-H-01] 레이싱 프로젝트 단일 청크 3.7MB (High)

현상: Vite 빌드 결과 index.js 가 3,761KB (gzip 1,307KB). Three.js + R3F + 게임 로직이 모두 단일 청크 위치: racing/dist/assets/index-*.js 영향: 초기 로딩 시간 3-5초 (3G 네트워크 기준 10초+) 권장: Three.js를 별도 청크로 분리 (dynamic import), 트랙 데이터 lazy loading 심각도: High

[BO-M-01] VirtuEx 프론트엔드 번들 — AI 분석 코드 메인 번들 포함 (Medium)

현상: AI 분석 모달 컴포넌트가 대시보드/뉴스 페이지에 인라인으로 포함되어 있어 해당 페이지 청크에 항상 포함 위치: frontend/src/app/(main)/dashboard/page.tsx:413-501 , news/page.tsx:370-498 영향: AI 분석을 사용하지 않는 사용자에게도 모달 코드 전달 권장: React.lazy 로 모달 컴포넌트 동적 임포트 심각도: Medium

[BO-L-01] Google Fonts 런타임 로딩 — FOUT 발생 (Low)

현상: HUD 컴포넌트에서 @import url('https://fonts.googleapis.com/...') 방식으로 폰트 로딩. 네트워크 의존적 위치: racing/src/game/ui/HUD.tsx:407 영향: 오프라인 시 폰트 로딩 실패, FOUT 발생 권장: 폰트 자체 호스팅

또는 @fontsource 패키지 사용 심각도: Low

종합 의견

3차 성능 감사에서는 총 35건의 이슈가 발견되었습니다. Critical 1건(리더보드 전체 계좌 스캔)은 사용자 증가에 따른 서버 부하 확대가 예상되므로 캐싱 전략 도입이 필요합니다.

2차 대비 개선된 점:

- WebSocket 배치 처리 도입으로 대시보드 렌더링 안정성 향상
- 시장가격 인메모리 캐시 적용으로 중복 외부 API 호출 감소
- 텔레메트리 객체 재사용으로 레이스 GC 부담 최소화

주요 신규 발견:

- 레이스 고가도로 마킹 이중 렌더링 (6개 추가 트랙으로 영향 확대)
- 리더보드 전체 계좌 스캔 + 10초 폴링 조합의 서버 부하
- 단일 번들 3.7MB — 코드 분할 필요

본 보고서는 자동 생성된 감사 결과이며, 수정 사항은 포함되지 않습니다.